

Architecting Ultra-Low Latency High-Frequency Trading Systems: Integrating Deep Reinforcement Learning, Project Panama, and Kernel Bypass

Introduction: The Convergence of Intelligence and Microsecond Determinism

The landscape of High-Frequency Trading (HFT) and algorithmic market making is currently undergoing a profound architectural evolution. Historically, quantitative trading systems were rigidly bifurcated. Researchers and quants utilized high-level languages like Python to model statistical arbitrages and stochastic control frameworks, while core execution systems were written exclusively in C or C++ to achieve deterministic, microsecond-level access to network hardware. However, the introduction of Deep Reinforcement Learning (DRL) into live trading environments has necessitated a paradigm shift. Financial institutions require the adaptive, non-linear predictive power of deep neural networks without sacrificing the ultra-low latency guarantees of their execution engines.

Bridging the gap between a Python-trained DRL model and a C++ execution engine presents immense serialization and inter-process communication (IPC) bottlenecks. Deploying a Python runtime directly into the critical path is strictly prohibited in HFT due to the Global Interpreter Lock (GIL) and non-deterministic memory management characteristics¹. The modern architectural consensus relies on a hybrid technology stack: models are trained offline in Python, exported to interoperable formats such as the Open Neural Network Exchange (ONNX), and deployed into a managed, high-performance inference runtime. Java 22+, augmented by the Deep Java Library (DJL) and Spring Boot, has emerged as the premier orchestration and inference layer, bridging the gap between high-level machine learning and low-level execution². This comprehensive analysis investigates the optimization of a tri-lingual HFT architecture. It explores the foundational mechanics of DRL in market making, the implementation of kernel bypass networking in C++ to circumvent the Linux TCP/IP stack, the utilization of Project Panama's Foreign Function & Memory (FFM) API to achieve true zero-copy data transfer between C++ and Java, and the precise tuning of Java's Generational Z Garbage Collector (ZGC) to guarantee sub-millisecond pauses under extreme tensor allocation loads.

Deep Reinforcement Learning in High-Frequency Market Making

High-frequency market making is an algorithmic trading strategy characterized by the

continuous posting of passive bid and ask quotes to capture the bid-ask spread⁶. The objective is to provide liquidity to the market while systematically managing the risks of adverse selection and inventory accumulation.

Stochastic Foundations and the Shift to DRL

Traditional market-making architectures rely heavily on stochastic control theory, most notably the Avellaneda-Stoikov model. In these classical frameworks, order arrivals are mathematically modeled as Poisson processes, where the intensity of the arrival decays exponentially as the distance of the quote from the stochastic mid-price increases⁷. Market makers adjust their quotes dynamically to balance the expected profit from spread capture against the inventory risk associated with holding a directional position in the asset⁷.

However, modern financial markets are highly non-stationary environments, characterized by complex microstructure dynamics such as semi-Markov price processes, Hawkes Jump-Diffusion dynamics, and shifting order flow imbalances⁷. Fixed heuristic models struggle to adapt to these rapid regime changes. Deep Reinforcement Learning addresses this limitation by allowing an autonomous agent to learn optimal quoting strategies directly from historical and simulated limit order book data.

State Representation, Architecture, and Reward Formulation

In a DRL market-making system, the environment is defined by the high-dimensional state of the Level-2 Limit Order Book (LOB). The agent observes the current state—which includes bid and ask depths, momentum indicators, volatility metrics, and its own current inventory—and executes actions that dictate the pricing and sizing of its limit orders⁶.

Advanced architectures frequently employ variants of Deep Q-Networks (DQN). Specifically, the Dueling Double Deep Q-Network (D3QN) has demonstrated superior performance in market-making simulations by mitigating the overestimation bias inherent in standard DQNs⁶. The D3QN separates the estimation of the state-value function from the advantage function, allowing the agent to more accurately evaluate the intrinsic value of a market state regardless of the specific quoting action taken⁶.

The success of the DRL agent is entirely dependent on the mathematical formulation of its reward function. Utilizing a naive mark-to-market Profit and Loss (PnL) calculation is highly problematic. In scenarios where the order flow is extremely unbalanced, a pure PnL reward incentivizes the agent to abandon market making and behave as a directional speculator, holding large inventories to ride price trends⁶. To enforce pure liquidity provision, the reward function must explicitly penalize inventory accumulation. An optimal continuous reward function is typically formulated to subtract an inventory penalty from the realized and unrealized trading profit. This ensures the agent maintains a mean-reverting inventory control strategy, quoting asymmetrically to offload positions when its inventory deviates significantly from zero⁶.

Circumventing the Kernel: C++ Execution and Network Bypass

For the DRL agent to interact with the exchange, the underlying C++ execution engine must process market data and transmit orders with absolute minimal latency. The standard Linux networking stack is a severe bottleneck in this endeavor.

When a network packet arrives at a standard Network Interface Card (NIC), it triggers a hardware interrupt. The operating system forces a context switch, copies the packet from the NIC hardware into a kernel-space socket buffer, processes the data through the highly generalized and complex TCP/IP stack, and finally copies the payload into the user-space application buffer upon the invocation of a `recv()` system call¹⁰. This traditional traversal consumes tens of microseconds, pollutes the CPU cache, and restricts throughput to roughly 1 million packets per second—introducing latency jitter that renders sub-millisecond HFT impossible¹⁰.

Kernel bypass technologies resolve this by eliminating the operating system from the data path entirely. By mapping the NIC's memory directly into the user-space application, the C++ execution engine can poll the hardware directly, achieving zero-copy I/O, eliminating context switches, and reducing network latency to the low single-digit microsecond range¹¹.

Evaluating Kernel Bypass Frameworks

When integrating kernel bypass into a C++ shared library (.so) designed to feed a Java DRL engine, architects must select between hardware-agnostic frameworks and vendor-specific APIs.

The Data Plane Development Kit (DPDK)

Maintained by the Linux Foundation, DPDK is a hardware-agnostic framework that provides user-space libraries and drivers for rapid packet processing¹¹. DPDK utilizes an Environment Abstraction Layer (EAL) and Poll Mode Drivers (PMD). Instead of relying on interrupts, the C++ application executes a continuous busy-wait loop (`rte_eth_rx_burst`), polling the NIC's receive queues¹⁰. Packets are transferred via Direct Memory Access (DMA) into pre-allocated memory pools (mbufs) mapped to CPU hugepages, ensuring zero-copy access¹⁰.

While DPDK offers extraordinary throughput for raw Layer 2/3 packet processing, it presents severe limitations for HFT order execution. DPDK inherently lacks a native TCP/IP stack¹⁰. Because order routing protocols (such as the FIX protocol) require reliable TCP connections, utilizing DPDK forces the engineering team to integrate and maintain a separate user-space TCP stack (e.g., mTCP, Seastar, or F-Stack) on top of the DPDK abstraction¹⁰. This introduces massive development complexity and operational risk into the C++ .so library.

The Solarflare Ecosystem: OpenOnload, TCPDirect, and ef_vi

For financial messaging, Solarflare (now AMD Xilinx) network adapters dominate the industry. Solarflare provides a tiered software ecosystem that allows engineers to select the optimal balance between ease of integration and absolute latency¹².

1. **OpenOnload:** This is a dynamically linked userspace library that provides a fully RFC-compliant TCP/UDP stack. It intercepts standard POSIX socket API calls via `LD_PRELOAD`¹⁰. The primary advantage of OpenOnload is absolute transparency; legacy

C++ applications require no source code modifications to achieve kernel bypass¹⁶. However, because it must maintain strict POSIX compliance and support blocking semantics, it carries inherent overhead¹².

2. **ef_vi (Ethernet Frame Virtual Interface):** This is Solarflare's lowest-level Layer 2 API¹². It grants the C++ application direct, fine-grained control over the NIC's ring buffers. ef_vi allocates hidden receive queues on the NIC and requires the application to manually construct raw ethernet frames¹⁵. While it provides the absolute lowest latency floor, it requires the application to implement its own ARP, IP, and TCP state machines, rendering it excessively complex for standard order routing¹⁷.
3. **TCPDirect:** TCPDirect serves as the optimal architectural compromise for HFT execution engines. It is a proprietary API that implements a lightweight, highly optimized TCP/IP stack entirely in user space¹². Unlike OpenOnload, TCPDirect drops the requirement for POSIX compliance, exposing an asynchronous, event-driven API¹². Under the hood, TCPDirect leverages ef_vi mechanisms but abstracts the complexity of TCP state management¹².

The performance differentials between these approaches are critical. Benchmark testing of the CME iLink 3 Binary Order Entry protocol illustrates the diminishing latency returns as one moves closer to the hardware metal.

Networking Stack	Handler Receive (99th Pct)	Handler Send (99th Pct)	Send + Receive (99th Pct)	OS + Handler Send (99th Pct)
Native Linux Sockets	84 µs	60 µs	132 µs	3639 µs
Solarflare OpenOnload	73 µs	62 µs	124 µs	693 µs
Solarflare TCPDirect	34 µs	56 µs	82 µs	297 µs

Table 1: Latency performance benchmarks across network stacks executing the CME iLink 3 protocol. Measurements capture the 99th percentile latency in microseconds (µs)¹⁸.

As demonstrated, replacing Native Sockets with OpenOnload reduces the 99th percentile OS + Handler Send latency from over 3.6 milliseconds to 693 microseconds. Moving to the proprietary TCPDirect API further slashes this to 297 microseconds, while tightening the variance band to essentially eliminate systemic jitter¹⁸.

C++ Library Implementation Architecture

To optimize the architecture, the C++ execution engine is compiled as a shared library (.so) that leverages TCPDirect for executing trades over TCP, and potentially raw ef_vi for ingesting UDP multicast market data feeds.

The application architecture initializes an instance of the TCPDirect::Stack. Configuration attributes dictate behavior, such as binding the stack to specific hardware interfaces using environment variables (e.g., ZF_ATTR="interface=eth4")¹⁹. The C++ engine creates endpoint sessions and registers a SessionListener¹⁹. Instead of blocking on read calls, the execution thread enters a continuous busy-wait loop, calling stack.dispatchEvents() to poll the hardware¹⁹. To maximize the efficacy of this polling loop, the Linux environment must be meticulously tuned. The CPU cores executing the C++ library must be isolated using kernel parameters (isolcpus), keeping the OS scheduler from interrupting the busy-wait loops¹⁰. Furthermore, power management states (C-states and P-states) must be disabled in the BIOS to prevent the CPU from entering energy-saving modes that delay instruction execution¹³.

Zero-Copy Interoperability: Project Panama and the FFM API

As the C++ execution engine ingests market data via TCPDirect, it constructs a dense, multi-level Limit Order Book. This massive data structure represents the state space that must be evaluated by the Java-based DRL inference engine.

Historically, passing complex structures from native C++ code to the Java Virtual Machine required the Java Native Interface (JNI). JNI is notoriously inefficient for high-frequency applications. Transferring a LOB array requires expensive data marshalling, copying the data from native memory into the Java heap²⁰. Furthermore, JNI requires developers to write fragile C-glue code; any pointer mismanagement or segmentation fault in the native code will instantly crash the entire JVM process²⁰.

While Java developers previously attempted to circumvent JNI using java.nio.ByteBuffer and the internal sun.misc.Unsafe class, these approaches were fundamentally flawed. ByteBuffer indices are limited to 32-bit integers, restricting maximum capacity to 2GB, and they rely on non-deterministic garbage collection for the deallocation of native memory²⁰.

The Foreign Function & Memory (FFM) API

Project Panama, finalized in Java 22 (JEP 454), entirely replaces JNI. The Foreign Function & Memory (FFM) API provides a safe, highly optimized, and purely Java-based mechanism for allocating native memory, mapping C-struct layouts, and invoking native functions²⁰.

The zero-copy LOB transfer pipeline relies on a specific set of FFM abstractions designed to ensure spatial and temporal safety without sacrificing performance.

Memory Management: Arenas and Segments

The foundational concept of FFM is the separation of memory lifecycle management from the memory container itself.

- **Arena:** An Arena dictates the lifecycle and thread-safety bounds of native memory,

replacing the manual malloc and free paradigms of C²⁰. The FFM API provides multiple arena types. A Confined Arena restricts access to a single thread for maximum performance, while an Automatic Arena delegates cleanup to the garbage collector²⁰. For sharing the LOB between the C++ polling thread and the Java inference thread, a Shared Arena (Arena.ofShared()) is instantiated. This allows concurrent access and ensures memory is deterministically freed only when explicitly closed, preventing memory leaks²⁰.

- **MemorySegment:** A MemorySegment represents the contiguous block of raw memory, analogous to a highly secure C pointer²⁰. Segments enforce spatial safety by executing strict bounds checks—attempting to read past the allocated size results in a standard Java IndexOutOfBoundsException rather than a silent memory corruption²⁰. They also enforce temporal safety; accessing a segment after its parent Arena has been closed throws an IllegalStateException²⁰.

Layout Mapping and Pointer Dereferencing

To eliminate serialization, Java must understand the exact byte layout of the C++ structs populating the MemorySegment. The FFM API utilizes ValueLayout, GroupLayout, and SequenceLayout to achieve this.

If the C++ engine populates an array of order book levels (e.g., a struct containing a double price and an int32_t volume), the Java architecture models this natively:

```
Java
GroupLayout levelLayout = MemoryLayout.structLayout(
    ValueLayout.JAVA_DOUBLE.withName("price"),
    ValueLayout.JAVA_INT.withName("volume")
);
MemorySegment segment = MemorySegment.allocateNative(1000, levelLayout);
```

This code defines a contiguous memory layout representing 1,000 price levels, mirroring the C++ memory footprint exactly²².

To read and write to this shared memory at nanosecond speeds, FFM utilizes VarHandle objects derived directly from the GroupLayout²². A VarHandle acts as a precompiled, strongly typed memory accessor. Because the Java Just-In-Time (JIT) compiler deeply understands the FFM API, it aggressively optimizes VarHandle invocations, often compiling the memory read down to a single MOV machine instruction on x86-64 architectures²⁰.

Function Linkage (Downcalls and Upcalls)

To initialize the execution engine, Java must call functions within the C++ .so library. The

SymbolLookup interface locates the memory address of the C++ function (e.g., `start_tcpdirect_stack`)²⁰. The Linker then binds this raw memory address to a Java `MethodHandle` using a `FunctionDescriptor` that defines the expected C signatures²⁰. Because `MethodHandle` invocations are strongly typed, the JIT compiler can inline the native machine code directly into the Java execution path²⁰. Through this architecture, the C++ execution engine continuously writes LOB updates directly into a shared native memory segment. The Java DRL engine, upon receiving an event trigger, uses a `VarHandle` to read the exact floating-point values out of native memory and feed them directly into the inference tensor, achieving a mathematically perfect zero-copy state transfer.

Guaranteeing Sub-Millisecond Determinism: Generational ZGC Tuning

While Project Panama eliminates IPC latency, the internal operations of the Java Virtual Machine present a significant hazard. Feeding the zero-copy LOB data into the DRL neural network via the Deep Java Library (DJL) requires the continuous instantiation of intermediate object wrappers, `NDArrays`, and inference context objects⁵.

In traditional JVM architectures employing throughput-oriented collectors like Parallel GC or the Garbage-First Garbage Collector (G1GC), reclaiming this memory requires "Stop-The-World" (STW) pauses. The duration of these pauses scales linearly with the size of the heap and the application's allocation rate¹. In an HFT context, a multi-millisecond G1GC pause during a period of extreme market volatility—precisely when the DRL agent must execute—results in missed opportunities, adverse selection, and catastrophic financial loss²⁹.

The Mechanics of the Z Garbage Collector

The Z Garbage Collector (ZGC) represents a monumental engineering achievement within the JVM, designed explicitly to maintain STW pause times below 1 millisecond regardless of whether the heap is 500 megabytes or 16 terabytes³⁰. ZGC accomplishes this by executing virtually all garbage collection tasks—including root scanning, marking, relocation, and reference remapping—concurrently with the application threads³⁰.

ZGC relies on two core architectural pillars:

1. **Colored Pointers:** On 64-bit architectures, pointers contain unused bits. ZGC utilizes these bits to store metadata about the object's state (e.g., whether it has been marked for survival or is currently being relocated to avoid memory fragmentation)³⁰.
2. **Load Barriers:** The JIT compiler injects highly optimized, microscopic snippets of code (load barriers) every time an application thread attempts to read an object reference from the heap³⁰. If the ZGC background threads are currently moving an object, the load barrier intercepts the read, updates the reference to the object's new physical memory location, and allows the application thread to proceed safely³⁰.

With the release of Java 21/22, Oracle introduced Generational ZGC (JEP 439). Previous iterations of ZGC scanned the entire heap during every cycle. Generational ZGC partitions the heap into a young generation (for newly allocated objects) and an old generation (for long-lived

orchestration state)³². By exploiting the weak generational hypothesis—which posits that the vast majority of objects (such as DJL inference tensors) die almost immediately after creation—Generational ZGC can aggressively and efficiently reclaim memory from the young generation without scanning the entire heap, delivering up to a 20% throughput improvement and significantly lowering CPU overhead³².

Explicit JVM Tuning for Extreme Tensor Workloads

Despite its sophisticated design, Generational ZGC requires precise manual tuning to survive the extreme allocation rates of a high-frequency DRL inference engine. Relying on default JVM ergonomics will result in latency spikes.

JVM Command-Line Flag	Architectural Purpose in HFT Context
-XX:+UseZGC -XX:+ZGenerational	Explicitly enables the Z Garbage Collector and activates the Generational mode, separating short-lived tensor allocations from long-lived state to improve GC throughput ³¹ .
-Xms32g -Xmx32g	Locks the minimum and maximum heap sizes to identical values. This prevents the JVM from executing expensive, latency-inducing OS calls to dynamically resize the heap during market trading hours ³¹ .
-XX:-ZUncommit	Disables ZGC's default behavior of returning freed memory pages to the Linux operating system. In HFT, relinquishing memory saves footprint but forces the JVM to re-request memory during volume spikes, introducing severe latency ³¹ .
-XX:+AlwaysPreTouch	Forces the JVM to write a zero to every page of the configured heap during application startup. This ensures the Linux kernel physically maps the memory to RAM upfront, eliminating page fault latency during live trading ³¹ .
-XX:+UseLargePages	Instructs the JVM to utilize explicit OS large

	pages (e.g., 2MB pages). This significantly reduces Translation Lookaside Buffer (TLB) misses, accelerating memory access speeds. <i>Note: Transparent Huge Pages (THP) must be disabled at the OS level to avoid kernel defragmentation stalls</i> ³¹ .
-XX:ConcGCThreads=N	Explicitly configures the number of background threads ZGC uses for concurrent collection. This is the most critical tuning parameter to prevent allocation stalls during heavy inference ³⁰ .

Table 2: Essential JVM configuration flags for tuning Generational ZGC in ultra-low latency machine learning environments.

Mitigating the Allocation Stall Hazard

The single greatest threat to a ZGC-powered HFT system is the "allocation stall." Because ZGC operates concurrently, it requires dedicated CPU resources. If the application threads (running the DRL inference via ONNX) allocate new objects faster than the ZGC concurrent threads can reclaim dead memory, the heap will exhaust its available headroom³⁴. When this occurs, ZGC acts as a circuit breaker, halting the application threads and forcing them to assist with garbage collection until memory is freed³¹.

An allocation stall introduces catastrophic, multi-millisecond tail latencies. To mitigate this, the engineer must tune the -XX:ConcGCThreads parameter. By default, ZGC allocates approximately 25% of available CPU cores to concurrent collection³⁰. In a highly CPU-bound DRL workload, this may be insufficient. Architects must profile the application's allocation rate (GB/sec) and dedicate sufficient, isolated CPU cores to ZGC to ensure the collection rate permanently outpaces the allocation rate³⁰.

When tuned correctly, the performance delta between standard collectors and ZGC at the extreme tail percentiles is profound. While G1GC and ZGC may demonstrate comparable median latencies, ZGC consistently flattens the 99.9th (p999) and 99.99th (p9999) percentile latencies, eradicating the microsecond spikes that trigger missed executions²⁹.

Enterprise Orchestration: Deep Java Library and ONNX Integration

With the network bypassing the kernel, data flowing seamlessly into Java via FFM, and ZGC neutralizing garbage collection pauses, the architecture requires a framework to execute the trained DRL models.

While Python dominates the model training phase, utilizing tools like PyTorch or TensorFlow, deploying these models in production requires exporting them to the Open Neural Network Exchange (ONNX) format². ONNX serves as the interoperable "Esperanto" of machine learning,

allowing models trained in Python to be executed in high-performance C++ or Java environments³⁶.

The Deep Java Library (DJL), open-sourced by AWS, provides the optimal integration layer. DJL is an engine-agnostic framework; it provides a native Java API that abstracts the underlying execution engines (such as the ONNX Runtime, PyTorch C++ API, or Apache MXNet)².

Spring Boot Integration and Model Deployment

DJL integrates seamlessly with modern enterprise Java architectures via Spring Boot. By incorporating specific starter dependencies (e.g., `djl-spring-boot-starter-pytorch-auto` or the ONNX equivalent), the application automatically downloads the requisite native libraries for the host operating system during startup³⁷.

Within the Java application, developers define a Criteria object to dictate how the model is loaded. The Criteria API acts as a builder pattern, specifying the input and output data types, the target execution engine, and the model's location (which can be a local file path, an AWS S3 bucket, or a Model Zoo repository)⁵.

Once the model is loaded, DJL generates a Predictor. It is critical to note that while the underlying model is thread-safe, the Predictor instances are not. In a multi-threaded HFT environment processing parallel order book updates, the architecture must utilize a thread-safe ZooModel to spawn dedicated Predictor instances for each execution thread, ensuring that inference operations do not block one another³.

Talent Economics and the Quantitative Engineering Market

The synthesis of highly specialized domains—Linux kernel bypass networking, C++17 template metaprogramming, Java Virtual Machine internals, and Deep Reinforcement Learning—represents the absolute frontier of computational finance. Developing and maintaining this hybrid architecture requires a caliber of engineering talent that is exceedingly scarce.

Firms seeking to deploy these systems—including traditional proprietary trading desks, hedge funds, and a new wave of cryptocurrency market makers such as Wintermute, Keyrock, and GSR—are aggressively competing for this hybrid skill set⁴¹. The market for engineers capable of crossing the chasm between low-level infrastructure and AI/ML model deployment has driven compensation to historic highs.

Global Tech Hub	Software Engineer Base (Senior)	AI/ML Engineer Base (Senior)	Quantitative Developer / HFT Total Comp (Senior)
New York / San Francisco	\$220K – \$310K	\$220K – \$310K+	\$500K – \$1.2M+

Zurich, Switzerland	CHF 130K – 170K	CHF 150K – 200K	CHF 300K – 500K+
London, UK	£110K – £170K	£110K – £220K	£250K – £400K+
Amsterdam, Netherlands	€105K – €150K	€110K – €170K	€180K – €300K+

Table 3: Estimated senior-level compensation data (2025/2026) across major global tech hubs, illustrating the significant premium paid for AI/ML and Quantitative HFT engineering skills relative to standard software development⁴⁶. Total compensation figures include base salary, aggressive performance bonuses, and equity/profit-sharing.

As demonstrated, AI/ML specialization commands a sharp premium over standard software engineering, often adding 20% to 40% to base salaries⁵⁰. However, engineers who can apply AI techniques directly to low-latency trading infrastructure occupy a distinct compensation echelon. At elite trading firms (e.g., Optiver, Jane Street, or leading crypto market makers), total compensation packages for senior quantitative developers routinely exceed half a million dollars, driven by aggressive, performance-linked cash bonuses that dwarf base salaries⁴⁸.

This talent scarcity has also catalyzed a shift toward remote, borderless hiring. Cryptocurrency market makers, natively accustomed to decentralized operations, frequently recruit remote C++ and Rust engineers across European and US time zones to build and maintain their matching engines and automated quoting systems, offering top-tier US or London-band salaries to secure the necessary expertise regardless of geographic location⁴⁴.

Conclusion

The architecture analyzed in this report effectively reconciles the computational complexity of Deep Reinforcement Learning with the unyielding latency constraints of High-Frequency Trading. By strategically allocating responsibilities across a hybrid stack, the system eliminates traditional bottlenecks.

The integration of Solarflare's TCPDirect API in the C++ execution engine ensures that network I/O completely bypasses the Linux kernel, facilitating order routing at the absolute physical limits of the network interface card. Project Panama's Foreign Function & Memory API eradicates the overhead of the Java Native Interface, utilizing Arena lifecycle management and VarHandle JIT optimization to share memory safely and read complex C-structs at zero-copy speeds. Finally, meticulously tuning Java's Generational Z Garbage Collector ensures that the massive volume of temporary objects generated by DJL/ONNX inference is reclaimed concurrently, guaranteeing that the system remains responsive and deterministic at the 99.99th percentile.

This architectural synthesis provides a resilient, hyper-optimized foundation, empowering financial institutions to deploy autonomous, AI-driven market-making strategies directly into the

world's most competitive trading environments.

Works cited

1. Java 21 for AI: How Enterprise Teams Build ML-Ready Systems - Bluepes, <https://bluepes.com/blog/java-21-the-its-release-powering-modern-ai-ready-systems>
2. Java AI Frameworks in 2026 - Credo Systemz, <https://www.credosystemz.com/blog/java-ai-framework/>
3. Java-Based AI for Large Language Models (LLMs): Explore | by Tanvir Khan | Medium, <https://aeontanvir.medium.com/java-based-ai-for-large-language-models-llms-explore-b9f8d967204f>
4. The Gen AI Iceberg - Java Tooling Edition - JAVAPRO International, <https://javapro.io/2026/06/03/the-gen-ai-iceberg-java-tooling-edition/>
5. Getting Started with Deep Java Library (DJI): Architecture & Basics - NashTech Blog, <https://blog.nashtechglobal.com/getting-started-with-deep-java-library-dji-architecture-basics/>
6. Performance of Deep Reinforcement Learning for High Frequency Market Making on Actual Tick Data - IFAAMAS, <https://www.ifaamas.org/Proceedings/aamas2022/pdfs/p1765.pdf>
7. Reinforcement Learning Market Maker | by Rohan Azad - Medium, <https://medium.com/@rohan.azad.ibn/reinforcement-learning-market-maker-36642172b9d8>
8. Deep Reinforcement Learning for High-Frequency Market Making, <https://proceedings.mlr.press/v189/kumar23a/kumar23a.pdf>
9. Deep Reinforcement Learning in Non-Markov Market-Making - MDPI, <https://www.mdpi.com/2227-9091/13/3/40>
10. Kernel Bypass Technologies - Damon's Blog, <https://www.damonyuan.com/tech/260203-kernel-bypass-technologies>
11. Kernel Bypass Techniques in Linux for High-Frequency Trading: A Deep Dive - lambdafunc, <https://lambdafunc.medium.com/kernel-bypass-techniques-in-linux-for-high-frequency-trading-a-deep-dive-de347ccd5407>
12. What is kernel bypass and how is it used in trading? | Databento Microstructure Guide, <https://databento.com/microstructure/kernel-bypass>
13. Kernel Bypass in HFT: How to Reduce Latency in Linux - QuantVPS, <https://www.quantvps.com/blog/kernel-bypass-in-hft>
14. Solarflare TCPDirect - Data World Computer & Communication Ltd., <https://dwcc.dataworld.com.hk/product/solarflare-tcpdirect/>
15. Kernel bypass - The Cloudflare Blog, <https://blog.cloudflare.com/kernel-bypass/>
16. Using Solarflare OpenOnload to Achieve Extreme Low Latency on Red Hat Enterprise Linux 6, <https://access.redhat.com/node/217113/40/0/5812943>
17. difference between Xilinx solarflare scaleout onlod, solarflare onload, and

- solarflare TCPDirect? - Stack Overflow,
<https://stackoverflow.com/questions/77843158/difference-between-xilinx-solarflare-scaleout-onlod-solarflare-onload-and-sola>
18. OnixS CME iLink 3 SDK with SolarflareTM TCPDirect ultra-low Latency Network Stack,
<https://www.onixs.biz/insights/onixs-cme-ilink-3-sdk-with-solarflaretm-tcpdirect-ultra-low-latency-network-stack>
 19. OnixS C++ FIX Engine: Solarflare Specific Features,
https://ref.onixs.biz/cpp-fix-engine-guide/group__solarflare-specific-features.html
 20. Java Foreign Function & Memory API (Project Panama) | by Kaustubh Saha - Medium,
<https://medium.com/@kaustubh.saha/java-foreign-function-memory-api-project-panama-ebbc29f5daaf>
 21. Foreign Function & Memory API (FFM API) vs JNI: Straight Talk from the Field,
<https://amirgolmoradi.medium.com/foreign-function-memory-api-ffmpeg-api-vs-jni-straight-talk-from-the-field-d145116659ed>
 22. Guide to Java Project Panama - Baeldung,
<https://www.baeldung.com/java-project-panama>
 23. Project Panama in Java: Modern Native Interoperability Without JNI - NashTech Blog,
<https://blog.nashtechglobal.com/project-panama-in-java-modern-native-interoperability-without-jni/>
 24. Building a Storage Engine in Modern Java with Panama (Part 1) | belief driven design, <https://belief-driven-design.com/storage-from-scratch-01-602d1/>
 25. JEP 454: Foreign Function & Memory API - OpenJDK,
<https://openjdk.org/jeps/454>
 26. From JNI to FFM: The future of Java-native interoperability - IBM Developer,
<https://developer.ibm.com/articles/j-ffmpeg/>
 27. Java FFM - Foreign Function & Memory Access API (Project Panama) - roray.dev,
<https://www.roray.dev/blog/java-io-uring-ffmpeg/>
 28. panama-foreign/doc/panama_ffi.md at foreign-memaccess+abi - GitHub,
https://github.com/openjdk/panama-foreign/blob/foreign-memaccess%2Babi/doc/panama_ffi.md
 29. GC-TuneHFT: AI-Based Garbage Collection Optimization in High-Frequency Trading Environments,
<https://aijcs.org/index.php/aijcs/article/download/197/185/372>
 30. Understanding ZGC: The Low-Latency Garbage Collector in Java | by rupesh gupta,
<https://medium.com/@rupeshgupta0912/understanding-zgc-the-low-latency-garbage-collector-in-java-8664b0124263>
 31. 9 The Z Garbage Collector - Java - Oracle Help Center,
<https://docs.oracle.com/en/java/javase/21/gctuning/z-garbage-collector.html>
 32. Generational ZGC in Java - Baeldung,
<https://www.baeldung.com/java-21-generational-z-garbage-collector>
 33. zgc Template - Broadcom TechDocs,

- <https://techdocs.broadcom.com/us/en/vmware-tanzu/spring/tanzu-tc-server/10-1/tc-server/topics-templates-zgc.html>
34. Let's Take a Look at... Lower Java Tail Latencies With ZGC - Gunnar Morling, <https://www.morling.dev/blog/lower-java-tail-latencies-with-zgc/>
 35. Leveraging ONNX for Interoperable Machine Learning: A Practical Guide with Java Spring Boot | by kiarash shamaii | Medium, <https://medium.com/@kiarash.shamaii/leveraging-onnx-for-interoperable-machine-learning-a-practical-guide-with-java-spring-boot-67a2ffbe7bf1>
 36. JVM Iceberg - The AI edition - JAVAPRO International, <https://javapro.io/2025/12/11/jvm-iceberg-the-ai-edition/>
 37. deepjavalibrary/djl-spring-boot-starter - GitHub, <https://github.com/deepjavalibrary/djl-spring-boot-starter>
 38. Deep Java Library - Apache Camel, <https://camel.apache.org/components/4.18.x/djl-component.html>
 39. Adopting machine learning in your microservices with DJL (Deep Java Library) and Spring Boot | AWS Open Source Blog, <https://aws.amazon.com/blogs/opensource/adopting-machine-learning-in-your-microservices-with-djl-deep-java-library-and-spring-boot/>
 40. Model Loading - Deep Java Library, https://docs.djl.ai/master/docs/load_model.html
 41. Our Firm - QNT Partners, <https://qntpartners.com/firm>
 42. Wintermute - Remote, <https://remote.com/jobs/wintermute-c1lmwshf>
 43. Jobs at Wintermute: Explore current Opportunities - Wellfound, <https://wellfound.com/company/wintermute-trading/jobs>
 44. Remote Jobs at Keyrock (Jun 2026) - Web3 Jobs, <https://web3.career/web3-companies/keyrock+remote>
 45. GSR Jobs & Careers — 4 Open Positions (2026) | SailOnchain, <https://sailonchain.com/companies/gsr>
 46. Quant Jobs | Quantitative Analyst Jobs | Barclay Simpson, <https://www.barclaysimpson.com/specialisms/quant-jobs/>
 47. AI Engineer Salary Guide 2026: US, EU, Remote, by Level | AY Automate, <https://www.ayautomate.com/blog/ai-engineer-salary-guide-2026>
 48. Software Engineer Salary in Germany (2026) - Next Level Jobs EU, <https://nextleveljobs.eu/blog/software-engineer-salary/germany>
 49. 12 Companies Paying €100k+ in the Netherlands (2026), <https://nextleveljobs.eu/blog/companies-paying-100k/netherlands>
 50. European Tech Salaries 2026: Complete Guide for Companies & Candidates, <https://techstaq.io/european-tech-salaries-2026-the-complete-guide-for-companies-hiring-engineers-and-candidates-evaluating-offers/>
 51. 8 Companies Paying CHF 100k+ in Switzerland (2026), <https://nextleveljobs.eu/blog/companies-paying-100k/switzerland>
 52. Global Python & AI Engineering Talent Index 2026 - Uvik Software, <https://uvik.net/blog/global-python-ai-engineering-talent-index-2026/>
 53. Software Engineer Salary in the Netherlands (2026) - Next Level Jobs EU, <https://nextleveljobs.eu/blog/software-engineer-salary/netherlands>

- 54. Careers at Ondo Finance - Wintermute Ventures,
<https://vc-jobs.wintermute.com/jobs/ondo-finance>
- 55. Keyrock hiring Rust Engineer - Trading • Remote (Work from Home) | Himalayas,
<https://himalayas.app/companies/keyrock-com/jobs/rust-engineer-trading-4049934027>
- 56. Keyrock hiring Head of Engineering - Options • Remote (Work from Home) | Himalayas,
<https://himalayas.app/companies/keyrock-com/jobs/head-of-engineering-options>